

Performance Testing

Date	02 July 2026
Team ID	xxxxxx
Project Name	A Comprehensive Measure of Well-Being (HDI Prediction System)
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module / API	Login, Prediction, History
Test Environment	Hosted Server (Render)
Test Date	01 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home Page Loading	1	10	Page loads successfully
2	User Login	5	30	Login successful

3	HDI	5	30	Prediction generated correctly
4	View Prediction History	5	20	History displayed without errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 sec	Pass	Good performance
2	Response Time (Max)	< 5 seconds	2.8 sec	Pass	Within limit
3	Throughput (Req/sec)	10 Req/sec	12 Req/sec	Pass	Stable
4	Error Rate	< 1%	0%	Pass	No errors
5	CPU Utilization	< 80%	42%	Pass	Normal usage
6	Memory Utilization	< 80%	51%	Pass	Stable

Step 4: Observations & Analysis

- The application responded quickly under normal load.
- No request failures were observed during testing.
- Login and prediction modules performed efficiently.
- Overall application performance was stable.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Render Dashboard: A-Comprehensive-Measure-of-Well-Being-4

July 5, 2026 at 4:07 PM Live

[a974dee](#) Update requirements.txt

[https://a-comprehensive-measure-of-well-being-4.onrender.com](#)

Logs

Jul 5

```
04:11:15 PM [djb6l] https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
04:11:15 PM [djb6l] warnings.warn(
04:11:15 PM [djb6l] [2026-07-05 10:41:15 +0000] [58] [INFO] Starting gunicorn 20.0.0
04:11:15 PM [djb6l] [2026-07-05 10:41:15 +0000] [58] [INFO] Listening at: http://0.0.0.0:10200 (58)
04:11:15 PM [djb6l] [2026-07-05 10:41:15 +0000] [58] [INFO] Using worker: sync
04:11:15 PM [djb6l] [2026-07-05 10:41:15 +0000] [74] [INFO] Booting worker with pid: 74
04:11:15 PM [djb6l] [2026-07-05 10:41:15 +0000] [58] [INFO] Control socket listening at: /opt/render/.gunicorn/gunicorn.ctl
04:11:15 PM [djb6l] 127.0.0.1 - - [05/Jul/2026:10:41:15 +0000] "HEAD / HTTP/1.1" 200 0 "-" "Go-http-client/1.1"
04:11:15 PM [djb6l] ==> Your service is live 🎉
04:11:15 PM [djb6l] ==>
04:11:15 PM [djb6l] ==> Available at your primary URL https://a-comprehensive-measure-of-well-being-4.onrender.com
04:11:15 PM [djb6l] ==>
04:11:15 PM [djb6l] ==>
04:11:15 PM [djb6l] 127.0.0.1 - - [05/Jul/2026:10:41:16 +0000] "GET / HTTP/1.1" 200 3528 "-" "Go-http-client/2.0"
```

Need better ways to work with logs? Try the [Render CLI](#), [Render MCP Server](#), or set up a [log stream integration](#)

Render Dashboard: A-Comprehensive-Measure-of-Well-Being-4

July 5, 2026 at 4:07 PM Live

[a974dee](#) Update requirements.txt

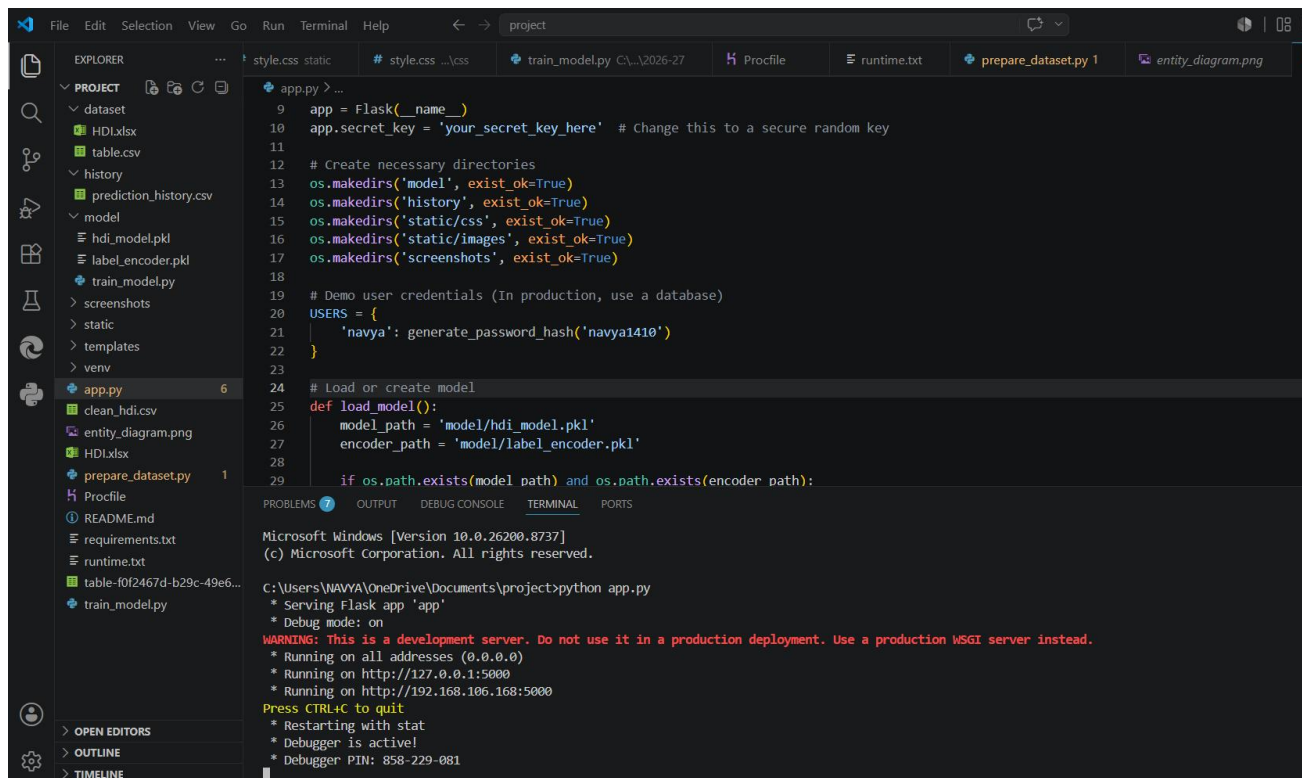
[https://a-comprehensive-measure-of-well-being-4.onrender.com](#)

Logs

Jul 5

```
04:08:40 PM [djb6l] Using cached werkzeug-3.1.6-py3-none-any.whl (226 kB)
04:08:40 PM [djb6l] Using cached packaging-26.2-py3-none-any.whl (100 kB)
04:08:40 PM [djb6l] Installing collected packages: threadpoolctl, six, packaging, numpy, narwhals, markupsafe, joblib, itsdangerous, click, blinker, werkzeug, scipy, python-dateutil, Jinja2, gunicorn, scikit-learn, pandas, Flask
04:09:07 PM [djb6l] Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.4.2 gunicorn-20.0.0 itsdangerous-2.2.0 Jinja2-3.1.6 joblib-1.5.3 markupsafe-3.0.3 narwhals-2.23.0 numpy-2.5.1 packaging-26.2 pandas-3.0.3 python-dateutil-2.9.0.post0 scikit-learn-1.9.0 scipy-1.18.0 six-1.17.0 threadpoolctl-3.6.0 werkzeug-3.1.8
04:09:07 PM [djb6l] [notice] A new release of pip is available: 25.3 -> 26.1.2
04:09:07 PM [djb6l] [notice] To update, run: pip install --upgrade pip
04:09:38 PM [djb6l] ==> Uploading build...
04:10:14 PM [djb6l] ==> Uploaded in 4.3s. Compression took 31.9s
04:10:14 PM [djb6l] ==> Build successful 🎉
04:10:25 PM [djb6l] ==> Deploying...
04:10:25 PM [djb6l] ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
04:10:30 PM [djb6l] ==> Running 'gunicorn app.py'
04:11:15 PM [djb6l] /opt/render/project/src/.venv/lib/python3.14/site-packages/sklearn/base.py:525: InconsistentVersionWarning: Trying to use pickle estimator
```

Need better ways to work with logs? Try the [Render CLI](#), [Render MCP Server](#), or set up a [log stream integration](#)



The screenshot shows a Visual Studio Code editor with a project named 'project'. The Explorer sidebar on the left shows the project structure, including files like 'dataset', 'HDL.xlsx', 'table.csv', 'history', 'prediction_history.csv', 'model', 'hdi_model.pkl', 'label_encoder.pkl', 'train_model.py', 'screenshots', 'static', 'templates', 'venv', 'app.py', 'clean_hdi.csv', 'entity_diagram.png', 'HDL.xlsx', 'prepare_dataset.py', 'Procfile', 'README.md', 'requirements.txt', 'runtime.txt', and 'table-f0f2467d-b29c-49e6...'. The main editor window displays the code for 'app.py', which is a Flask application. The code includes imports for Flask, os, and datetime, and defines a 'load_model()' function. The terminal at the bottom shows the command 'python app.py' being executed, resulting in a warning message and the application running on http://127.0.0.1:5000.

```

9  app = Flask(__name__)
10 app.secret_key = 'your_secret_key_here' # Change this to a secure random key
11
12 # Create necessary directories
13 os.makedirs('model', exist_ok=True)
14 os.makedirs('history', exist_ok=True)
15 os.makedirs('static/css', exist_ok=True)
16 os.makedirs('static/images', exist_ok=True)
17 os.makedirs('screenshots', exist_ok=True)
18
19 # Demo user credentials (In production, use a database)
20 USERS = {
21     'navya': generate_password_hash('navya1410')
22 }
23
24 # Load or create model
25 def load_model():
26     model_path = 'model/hdi_model.pkl'
27     encoder_path = 'model/label_encoder.pkl'
28
29     if os.path.exists(model_path) and os.path.exists(encoder_path):

```

Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\NAVYA\OneDrive\Documents\project>python app.py

* Serving Flask app 'app'

* Debug mode: on

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on all addresses (0.0.0.0)

* Running on http://127.0.0.1:5000

* Running on http://192.168.106.168:5000

Press CTRL+C to quit

* Restarting with stat

* Debugger is active!

* Debugger PIN: 858-229-081